NAME : DARSSHINI BABU RAJU
ROLL NO : 2025115144

1)Simulate the undo and redo option in word packages using a doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char text[100];
    struct Node *prev, *next;
};

struct Node *current = NULL;

void save(char text[]) {
    struct Node *newnode = malloc(sizeof(struct Node));
    strcpy(newnode->text, text);
    newnode->prev = current;
    newnode->next = NULL;

    if (current != NULL)
        current->next = newnode;

    current = newnode;
}

void undo() {
    if (current != NULL && current->prev != NULL)
        current = current->prev;
    printf("Text: %s\n", current->text);
}

void redo() {
    if (current != NULL && current->next != NULL)
        current = current->next;
    printf("Text: %s\n", current->text);
```

```
}

int main() {
    save("Hello");
    save("Hello World");
    save("Hello World!");

    printf("Current text: %s\n", current->text);

    printf("Undo:\n");
    undo();

    printf("Redo:\n");
    redo();

    return 0;
}
```

OUTPUT :

```
Current text: Hello World!
Undo:
Text: Hello World
Redo:
Text: Hello World!
```

2) Given two sorted lists L1 and L2 write a procedure to compute L1∩L2 and L1∪L2 using basic list operations

```
#include <stdio.h>

int main() {
    int a[50], b[50], u[100], in[50];
    int n, m, i = 0, j = 0, k = 0, p = 0;

    printf("Enter size of L1: ");
    scanf("%d", &n);

    printf("Enter sorted elements of L1: ");
```

```
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter size of L2: ");
    scanf("%d", &m);

    printf("Enter sorted elements of L2: ");
    for (i = 0; i < m; i++)
        scanf("%d", &b[i]);

    i = j = 0;

    while (i < n && j < m) {
        if (a[i] < b[j]) {
            u[k++] = a[i++];
        } else if (a[i] > b[j]) {
            u[k++] = b[j++];
        } else {
            in[p++] = a[i];
            u[k++] = a[i];
            i++;
            j++;
        }
    }

    while (i < n) u[k++] = a[i++];
    while (j < m) u[k++] = b[j++];

    printf("Intersection: ");
    for (i = 0; i < p; i++)
        printf("%d ", in[i]);

    printf("\nUnion: ");
    for (i = 0; i < k; i++)
        printf("%d ", u[i]);

    return 0;
}
```

OUTPUT :

```
Enter size of L1: 4
Enter sorted elements of L1: 1 2 4 6
Enter size of L2: 4
Enter sorted elements of L2: 2 3 4 5
Intersection: 2 4
Union: 1 2 3 4 5 6
```

3) Swap two elements by adjusting only the pointers (and not the data) using doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    n->prev = NULL;

    if (head == NULL) {
        head = n;
        return;
    }

    struct Node *t = head;
    while (t->next != NULL)
        t = t->next;

    t->next = n;
    n->prev = t;
}
```

```
void display() {
    struct Node *t = head;
    while (t != NULL) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

void swapNodes(struct Node *x, struct Node *y) {
    if (x == y) return;

    if (x->next == y) {
        struct Node *p = x->prev;
        struct Node *n = y->next;

        if (p) p->next = y;
        else head = y;

        if (n) n->prev = x;

        y->prev = p;
        y->next = x;
        x->prev = y;
        x->next = n;
        return;
    }

    if (y->next == x) {
        swapNodes(y, x);
        return;
    }

    struct Node *xp = x->prev, *xn = x->next;
    struct Node *yp = y->prev, *yn = y->next;

    if (xp) xp->next = y;
    else head = y;
```

```
    if (xn) xn->prev = y;

    if (yp) yp->next = x;
    else head = x;

    if (yn) yn->prev = x;

    x->prev = yp;
    x->next = yn;
    y->prev = xp;
    y->next = xn;
}

int main() {
    struct Node *x, *y, *t;

    insert(10);
    insert(20);
    insert(30);
    insert(40);

    x = head;
    y = head->next->next;

    printf("Before swapping: ");
    display();

    swapNodes(x, y);

    printf("After swapping: ");
    display();

    return 0;
}
```

OUTPUT :

```
Before swapping: 10 20 30 40
After swapping: 30 20 10 40
```

4) write an array implementation of self adjusting list. A self adjusting list is like a regular list, except all insertions are performed at the front and when an element is accessed by a find, it is moved to the front of the list, without changing the relative order of the other elements.

```
#include <stdio.h>

int a[50], n = 0;

void insertFront(int x) {
    int i;
    for (i = n; i > 0; i--)
        a[i] = a[i - 1];

    a[0] = x;
    n++;
}

void find(int x) {
    int i, j;

    for (i = 0; i < n; i++) {
        if (a[i] == x) {
            for (j = i; j > 0; j--)
                a[j] = a[j - 1];

            a[0] = x;
            printf("%d found and moved to front.\n", x);
            return;
        }
    }

    printf("%d not found.\n", x);
}

void display() {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
```

```
    printf("\n");
}

int main() {
    insertFront(10);
    insertFront(20);
    insertFront(30);

    printf("List: ");
    display();

    find(10);

    printf("Updated list: ");
    display();

    return 0;
}
```

OUTPUT :

```
List: 30 20 10
10 found and moved to front.
Updated list: 10 30 20
```

5) write a C program to reverse a given doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
    struct Node *n = malloc(sizeof(struct Node));
```

```
    n->data = x;
    n->next = NULL;
    n->prev = NULL;

    if (head == NULL) {
        head = n;
        return;
    }

    struct Node *t = head;
    while (t->next != NULL)
        t = t->next;

    t->next = n;
    n->prev = t;
}

void reverse() {
    struct Node *t = head, *temp = NULL;

    while (t != NULL) {
        temp = t->prev;
        t->prev = t->next;
        t->next = temp;
        t = t->prev;
    }

    if (temp != NULL)
        head = temp->prev;
}

void display() {
    struct Node *t = head;
    while (t != NULL) {
        printf("%d ", t->data);
        t = t->next;
    }
}

int main() {
```

```
    insert(10);
    insert(20);
    insert(30);
    insert(40);

    printf("Original list: ");
    display();

    reverse();

    printf("\nReversed list: ");
    display();

    return 0;
}
```

OUTPUT :

Original list: 10 20 30 40
Reversed list: 40 30 20 10

6) write a C program to delete the middle element of a DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    n->prev = NULL;

    if (!head) {
```

```
        head = n;
        return;
    }

    struct Node *t = head;
    while (t->next) t = t->next;
    t->next = n;
    n->prev = t;
}

void deleteMiddle() {
    struct Node *slow = head, *fast = head;

    if (!head) return;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow->prev)
        slow->prev->next = slow->next;
    else
        head = slow->next;

    if (slow->next)
        slow->next->prev = slow->prev;

    free(slow);
}

void display() {
    struct Node *t = head;
    while (t) {
        printf("%d ", t->data);
        t = t->next;
    }
}

int main() {
```

```
    insert(10);
    insert(20);
    insert(30);
    insert(40);
    insert(50);

    printf("Original list: ");
    display();

    deleteMiddle();

    printf("\nAfter deleting middle: ");
    display();

    return 0;
}
```

OUTPUT :

Original list: 10 20 30 40 50
After deleting middle: 10 20 40 50

7) Remove duplicate elements from a (i) sorted DLL (ii) unsorted DLL

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

void insert(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->next = NULL;
    n->prev = NULL;
```

```
    if (!head) {
        head = n;
        return;
    }

    struct Node *t = head;
    while (t->next) t = t->next;
    t->next = n;
    n->prev = t;
}

void removeDuplicates() {
    struct Node *p, *q, *temp;

    for (p = head; p != NULL; p = p->next) {
        q = p->next;

        while (q != NULL) {
            if (q->data == p->data) {
                temp = q;
                q = q->next;

                if (temp->prev)
                    temp->prev->next = temp->next;

                if (temp->next)
                    temp->next->prev = temp->prev;

                free(temp);
            } else {
                q = q->next;
            }
        }
    }
}

void display() {
    struct Node *t = head;
    while (t) {
        printf("%d ", t->data);
```

```
        t = t->next;
    }
}

int main() {
    insert(10);
    insert(20);
    insert(10);
    insert(30);
    insert(20);

    printf("Original list: ");
    display();

    removeDuplicates();

    printf("\nAfter removing duplicates: ");
    display();

    return 0;
}
```

OUTPUT :

```
Original list: 10 20 10 30 20
After removing duplicates: 10 20 30
```

8)Calculate the frequency count of a given integer in a circular DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;
```

```
void insert(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;

    if (head == NULL) {
        head = n;
        n->next = n;
        n->prev = n;
    } else {
        struct Node *last = head->prev;
        n->next = head;
        n->prev = last;
        last->next = n;
        head->prev = n;
    }
}

int frequency(int x) {
    int count = 0;
    struct Node *t = head;

    if (head == NULL) return 0;

    do {
        if (t->data == x)
            count++;
        t = t->next;
    } while (t != head);

    return count;
}

int main() {
    int x;

    insert(10);
    insert(20);
    insert(10);
    insert(30);
    insert(10);
```

```
    printf("Enter element: ");
    scanf("%d", &x);

    printf("Frequency of %d = %d\n", x, frequency(x));

    return 0;
}
```

OUTPUT :

```
Enter element: 10
Frequency of 10 = 3
```

9) Search for a particular element in a binary tree.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

int search(struct Node *root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) ||
```

```
        search(root->right, key);
}

int main() {
    int key;

    struct Node *root = create(10);
    root->left = create(20);
    root->right = create(30);
    root->left->left = create(40);
    root->left->right = create(50);

    printf("Enter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element found.\n");
    else
        printf("Element not found.\n");

    return 0;
}
```

OUTPUT :

```
Enter element to search: 40
Element found.
```

10)Calculate the height or depth of a tree and print it.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *create(int x) {
```

```
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

int height(struct Node *root) {
    int leftHeight, rightHeight;

    if (root == NULL)
        return 0;

    leftHeight = height(root->left);
    rightHeight = height(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;

    return rightHeight + 1;
}

int main() {
    struct Node *root = create(10);
    root->left = create(20);
    root->right = create(30);
    root->left->left = create(40);

    printf("Height of tree = %d\n", height(root));

    return 0;
}
```

OUTPUT :

Height of tree = 3

11) Given a binary tree, convert this to a doubly linked list keeping the same order (inorder traversal).

```
#include <stdio.h>
```

```
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *head = NULL, *prev = NULL;

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

void convert(struct Node *root) {
    if (root == NULL) return;

    convert(root->left);

    if (prev == NULL)
        head = root;
    else {
        root->left = prev;
        prev->right = root;
    }

    prev = root;

    convert(root->right);
}

void display() {
    struct Node *t = head;

    while (t != NULL) {
        printf("%d ", t->data);
        t = t->right;
    }
```

```
}

int main() {
    struct Node *root = create(20);
    root->left = create(10);
    root->right = create(30);
    root->left->left = create(5);
    root->left->right = create(15);

    convert(root);

    printf("Doubly linked list: ");
    display();

    return 0;
}
```

OUTPUT :
Doubly linked list: 5 10 15 20 30

12) Given a binary tree, connect the nodes that are at the same level using next right pointers.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right, *nextRight;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = n->nextRight = NULL;
    return n;
}

void connect(struct Node *root) {
```

```
    struct Node *q[100];
    int front = 0, rear = 0, count, i;

    if (root == NULL) return;

    q[rear++] = root;

    while (front < rear) {
        count = rear - front;

        for (i = 0; i < count; i++) {
            struct Node *t = q[front++];

            if (i < count - 1)
                t->nextRight = q[front];
            else
                t->nextRight = NULL;

            if (t->left) q[rear++] = t->left;
            if (t->right) q[rear++] = t->right;
        }
    }
}

int main() {
    struct Node *root = create(1);
    root->left = create(2);
    root->right = create(3);
    root->left->left = create(4);
    root->left->right = create(5);

    connect(root);

    printf("Node 2 next right: %d\n",
           root->left->nextRight->data);
    printf("Node 4 next right: %d\n",
           root->left->left->nextRight->data);
    printf("Node 3 next right: %s\n",
           root->right->nextRight == NULL ? "NULL" : "Connected");
```

```
    return 0;
}
```

OUTPUT :

```
Node 2 next right: 3
Node 4 next right: 5
Node 3 next right: NULL
```

13) Implement priority Queue using binary search tree and perform operations Insert, search and delete.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

struct Node *insert(struct Node *root, int x) {
    if (root == NULL) return create(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    return root;
}

int search(struct Node *root, int x) {
```

```
    if (root == NULL) return 0;
    if (root->data == x) return 1;

    if (x < root->data)
        return search(root->left, x);

    return search(root->right, x);
}

struct Node *minimum(struct Node *root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node *deleteNode(struct Node *root, int x) {
    struct Node *temp;

    if (root == NULL) return NULL;

    if (x < root->data)
        root->left = deleteNode(root->left, x);
    else if (x > root->data)
        root->right = deleteNode(root->right, x);
    else {
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }

        temp = minimum(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
```

```
    }

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;

    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 10);

    printf("Priority queue: ");
    inorder(root);

    printf("\nSearch 20: %s",
           search(root, 20) ? "Found" : "Not found");

    root = deleteNode(root, minimum(root)->data);

    printf("\nAfter deleting highest priority: ");
    inorder(root);

    return 0;
}
```

OUTPUT :

```
Priority queue: 10 20 30 40
Search 20: Found
After deleting highest priority: 20 30 40
```

14) In a given binary search tree write routine to delete a node with no children, one child and two children.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

struct Node *insert(struct Node *root, int x) {
    if (root == NULL) return create(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    return root;
}

struct Node *minNode(struct Node *root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node *deleteNode(struct Node *root, int x) {
    struct Node *temp;
```

```
    if (root == NULL) return NULL;

    if (x < root->data)
        root->left = deleteNode(root->left, x);
    else if (x > root->data)
        root->right = deleteNode(root->right, x);
    else {
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }

        temp = minNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void inorder(struct Node *root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
```

```
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Original BST: ");
    inorder(root);

    root = deleteNode(root, 20);
    printf("\nAfter deleting leaf node 20: ");
    inorder(root);

    root = deleteNode(root, 30);
    printf("\nAfter deleting node 30: ");
    inorder(root);

    root = deleteNode(root, 50);
    printf("\nAfter deleting node 50: ");
    inorder(root);

    return 0;
}
```

OUTPUT :

```
Original BST: 20 30 40 50 60 70 80
After deleting leaf node 20: 30 40 50 60 70 80
After deleting node 30: 40 50 60 70 80
After deleting node 50: 40 60 70 80
```

15) Convert the given postfix expression as expression tree.

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node {
    char data;
    struct Node *left, *right;
};
```

```
struct Node *stack[100];
int top = -1;

struct Node *create(char x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

void push(struct Node *n) {
    stack[++top] = n;
}

struct Node *pop() {
    return stack[top--];
}

void inorder(struct Node *root) {
    if (root) {
        if (root->left) printf("(");
        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);
        if (root->right) printf(")");
    }
}

void preorder(struct Node *root) {
    if (root) {
        printf("%c ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root) {
        postorder(root->left);
```

```
        postorder(root->right);
        printf("%c ", root->data);
    }
}

int main() {
    char exp[100], ch;
    int i;
    struct Node *n, *root;

    printf("Enter postfix expression: ");
    scanf("%99s", exp);

    for (i = 0; exp[i] != '\0'; i++) {
        ch = exp[i];

        if (isalnum((unsigned char)ch)) {
            push(create(ch));
        } else {
            n = create(ch);
            n->right = pop();
            n->left = pop();
            push(n);
        }
    }

    root = pop();

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    return 0;
}
```

OUTPUT :

```
Enter postfix expression: ab+c*
Inorder: ((a+b)*c)
Preorder: * + a b c
Postorder: a b + c *
```

16) Construct binary search tree using (i) Inorder node representation (ii) Preorder and (iii) Post order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *create(int x) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = x;
    n->left = n->right = NULL;
    return n;
}

struct Node *insert(struct Node *root, int x) {
    if (root == NULL)
        return create(x);

    if (x < root->data)
        root->left = insert(root->left, x);
    else if (x > root->data)
        root->right = insert(root->right, x);

    return root;
}

void inorder(struct Node *root) {
    if (root) {
        inorder(root->left);
```

```
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node *root) {
    if (root) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct Node *root = NULL;
    int a[] = {40, 20, 60, 10, 30, 50, 70};
    int i;

    for (i = 0; i < 7; i++)
        root = insert(root, a[i]);

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    return 0;
}
```

OUTPUT :
Inorder: 10 20 30 40 50 60 70
Preorder: 40 20 10 30 60 50 70
Postorder: 10 30 20 50 70 60 40

17) Implement heap sort for a set of numbers.

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for (i = n - 1; i > 0; i--) {
        temp = a[0];
```

```
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main() {
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    heapSort(a, n);

    printf("Sorted elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

OUTPUT :

```
Enter number of elements: 7
Enter elements: 50 20 8 30 25 15 22
Sorted elements: 8 15 20 22 25 30 50
```